

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Wydzial Informatyki, Elektroniki i Telekomunikacji

Katedra Telekomunikacji

Praca dyplomowa
inżynierska

Analiza i implementacja zabezpieczeń protokołu BGP

Analysis and implementation of BGP security measures

Imię i nazwisko: Radosław Brońka

Kierunek studiów: Teleinformatyka

Opiekun pracy: dr inż. Jerzy Domżał

12 lipca 2015

Oświadczam, świadomy odpowiedzialności karnej za poświadczenie nieprawdy, że niniejszą pracę dyplomową wykonałem osobiście i samodzielnie i że nie korzystałem ze źródeł innych niż wymienione w pracy.

Streszczenie

FAMTAR (Flow-Aware Multi-Topology Adaptive Routing) to nowo zaproponowany mechanizm rutingu wielodrogowego i adaptacyjnego. Stanowi on próbę odpowiedzi na wady istniejących mechanizmów rutingu wielodrogowego i adaptacyjnego. W odróżnieniu od większości z nich, może on działać wraz z dowolnym protokołem rutingu wewnątrzdomenowego. Jest on w pełni rozproszony i nie jest podatny na tzw. zjawisko oscylacji tras. Jako rozwiązanie nowe, wymaga on jednak szczegółowych badań.

Celem pracy była implementacja rutera programowego pracującego według mechanizmu FAMTAR oraz zbadanie działania sieci zbudowanych z jego użyciem. Ta praca przedstawia szczegóły opracowania i implementacji rutera, jak również wyniki testów sieciowych.

Praca zaczyna się od wprowadzenia do problematyki rutingu IP. Szczegółowo wyjaśnione są problemy związane ze stosowaniem rutingu wielodrogowego i adaptacyjnego. Zaprezentowane zostały istniejące rozwiązania w tym zakresie. Następnie w pracy przedstawiono szczegółowo mechanizm FAMTAR. Dalej praca wyjaśnia szczegóły implementacji rutera. Uzasadniony zostaje wybór środowiska implementacyjnego (Linux i Click). Opisana jest konfiguracja Clicka rutera i nowe elementy stworzone na potrzeby projektu. Na koniec przedstawione są wyniki testów. Potwierdzają one prawidłową pracę rutera i skuteczność mechanizmu FAMTAR.

Abstract

FAMTAR (Flow-Aware Multi-Topology Adaptive Routing) is a newly proposed multipath and adaptive routing mechanism. It attempts to overcome the limitations of existing multipath and adaptive routing mechanisms. Unlike most of them, it can cooperate with any interior routing protocol. It is fully distributed and it is not vulnerable to path oscillations. However, as a new solution, it still requires a lot of testing.

The goal of this work was to implement a software IP router, which utilizes FAMTAR mechanism, and to test networks built with it. This dissertation presents development and implementation details, as well as results of network tests.

A general IP routing background is provided first. Concepts and problems associated with multipath and adaptive routing are explained in detail. Existing solutions are presented and assessed. Next, the FAMTAR mechanism is presented in detail. After that, the dissertation explains the implementation details. The choice of implementation environment (Linux and Click) is explained. Click's configuration of the router and newly created Click elements are described. Finally, the results of tests are presented. They confirm the correctness of router operation and effectiveness of the FAMTAR mechanism.

Spis treści

Spis treści **8**

1 Wprowadzenie **9**

2 Zagadnienia teoretyczne **11**
2.1 Protokół BGP 11
2.2 Zagrożenia dla rutingu międzydomenowego 11
2.2.1 Wycieki prefiksów 11
2.2.2 Kradzieże prefiksów 11
2.3 Mechanizmy zabezpieczające BGP 11
2.3.1 BGP Roles 11
2.3.2 RPKI 12
2.3.3 ASPA 12
2.3.4 Filtracja prefiksów 12

3 Użyte narzędzia **13**
3.1 EVE-NG 13
3.2 FRRouting 13
3.3 Krill 13
3.4 Routinator 14

4 Implementacja topologii **15**
4.1 Projekt sieci 15
4.2 Konfiguracja 15

5 Demonstracja podatności **19**
5.1 Route Leak 19
5.1.1 Type 1 leak 19
5.1.2 Type 2 leak 19
5.1.3 Type 3 leak 20
5.1.4 Type 4 leak 20
5.1.5 Type 5 leak 21
5.1.6 Type 6 leak 21
5.2 Prefix Hijack 23
5.2.1 Exact-Prefix Hijack 23

5.2.2 Sub-prefix Hijack 24
5.2.3 Forged Origin Exact-Prefix Hijack 25
5.2.4 Forged Origin Sub-prefix Hijack 26

6 Implementacja i analiza zabezpieczeń **29**
6.1 BGP Roles i atrybut Only To Customer 29
6.1.1 Konfiguracja i implementacja 29
6.1.2 Analiza skuteczności 30
6.2 RPKI 34
6.3 ASPA 34
6.4 Filtracja prefiksów 34

7 Wnioski **35**

8 Podsumowanie **37**

A Źródła pakietu Clicka FAMTAR **41**

Rozdział 1

Wprowadzenie

Mechanizm FAMTAR (Flow-Aware Multi-Topology Adaptive Routing) to nowe rozwiązanie zapewniające ruting wielodrogowy i adaptacyjny w sieciach IP. Został on zaproponowany w [**?**] przez opiekunów autora tej pracy. Stanowi on próbę odpowiedzi na wady istniejących mechanizmów rutingu wielodrogowego i adaptacyjnego. Może on współpracować z dowolnym protokołem rutingu wewnątrzdomenowego. Wśród jego zalet można wymienić także w pełni rozproszoną strukturę oraz brak podatności na tzw. oscylację tras.

Mechanizm FAMTAR wydaje się w tej chwili obiecującym rozwiązaniem problemu rutingu wielodrogowego i adaptacyjnego. Wprowadzenie go do powszechnego użytku pozwoliłoby na znaczne zwiększenie efektywności wykorzystania zasobów sieciowych. Jako rozwiązanie nowe, wymaga on jednak szczegółowych badań.

Celem tej pracy było, wobec tego, stworzenie rutera programowego realizującego koncepcję FAMTAR. Został on zaimplementowany w środowisku Click i pracuje pod kontrolą systemu operacyjnego Linux. Oprócz samej implementacji rutera, w ramach pracy przeprowadzone zostały testy sieci zbudowanych z jego użyciem.

Ta praca podzielona jest na sześć rozdziałów. Jej zakres, motywacja i struktura przedstawione zostały w rozdziale pierwszym.

Rozdział 2 prezentuje tło na jakim powstał mechanizm FAMTAR. Zawarto w nim wprowadzenie do podstaw rutingu IP, po którym przedstawiono problematykę rutingu wielodrogowego i adaptacyjnego. Następnie zawiera on przegląd istniejących rozwiązań w tym zakresie.

Rozdział 3 przedstawia koncepcję FAMTAR. Opisane są w nim zarówno jej podstawy, jak i rozszerzenia. Część zaprezentowanych tam mechanizmów powstała w wyniku badań autora tej pracy.

Rozdział 4 opisuje szczegóły implementacji rutera. Zaprezentowano w nim środowisko implementacyjne i uzasadniono jego wybór. Następnie opisano w nim szczegółowo kolejne komponenty rutera.

Rozdział 5 prezentuje wyniki testów zaimplementowanego rutera, przeprowadzonych w laboratorium sieciowym.

Ostatni rozdział zawiera podsumowanie pracy. Przedstawione są osiągnięte cele i rezultaty.

Rozdział 2

Zagadnienia teoretyczne

Poniższy rozdział ma za zadanie przedstawić teoretyczne zagadnienia kluczowe dla tematyki pracy. W pierwszym podrozdziale opisano podstawy działania protokołu BGP. Kolejna sekcja omawia zjawiska wycieków i kradzieży prefiksów, będących zagrożeniem dla poprawności rutingu międzydomenowego. Trzeci podrozdział zawiera opis wybranych metod zabezpieczeń protokołu BGP, stanowiących przedmiot badań: mechanizmu BGP Roles, RPKI, ASPA oraz filtrowania prefiksów.

2.1 Protokół BGP

Test

2.2 Zagrożenia dla rutingu międzydomenowego

Test

2.2.1 Wycieki prefiksów

Wycieki :(

2.2.2 Kradzieże prefiksów

Kradzieże :(

2.3 Mechanizmy zabezpieczające BGP

Test

2.3.1 BGP Roles

Role

2.3.2 RPKI

RPKI

2.3.3 ASPA

ASPA

2.3.4 Filtracja prefiksów

filtracja

Rozdział 3

Użyte narzędzia

Narzędzia użyte w części praktycznej

3.1 EVE-NG

Jako środowisko, w którym zbudowana została topologia, posłużył emulator EVE-NG Community uruchomiony jako maszyna wirtualna w VMware Workstation PRO. Wybór narzędzia podyktowany był przede wszystkim wydajnością emulacji środowiska sieciowego oraz możliwościami zarządzania stanem i monitorowania ruchu w laboratorium. EVE-NG umożliwia zapis stanu całej topologii i eksport konfiguracji, co pozwala na archiwizację oraz odtwarzanie sieci po przeprowadzeniu scenariusza badawczego. Platforma natywnie współpracuje z oprogramowaniem Wireshark, pozwalając na przesyłanie ruchu z interfejsu wirtualnego rutera do analizatora pakietów. Dostępny interfejs graficzny działający w przeglądarce internetowej i integracja z programem PuTTY usprawnia proces budowy sieci i wdrożenia konfiguracji.

3.2 FRRouting

FRRouting to otwartoźródłowy pakiet protokołów rutingu przeznaczony dla systemów operacyjnych z rodziny Linux, stanowiący rozwinięcie i kontynuację projektu Quagga. Oprogramowanie oferuje pełną implementację protokołu BGP oraz charakteryzuje się lekką architekturą, co pozwoliło na optymalizację i ograniczenie zasobów obliczeniowych niezbędnych do uruchomienia topologii. Ze względu na zbiór metod opisywanych w pracy wybrana została wersja deweloperska o numerze 10.9.0-dev. Zapewnia ona wsparcie dla protokołu RTRv2, niezbędnego do działania mechanizmu ASPA, którego starsze, stabilne wersje nie posiadają.

3.3 Krill

Krill to demon RPKI autorstwa organizacji NLnet Labs, pełniący funkcję urzędu certyfikacji i serwera publikacji. Krill umożliwia uruchomienie środowiska testowego

i odizolowanie go od zewnętrznych rejestrów kontrolowanych przez RIR oraz natywnie wspiera tworzenie obiektów ASPA i zarządzanie nimi, co jest konieczne do przeprowadzenia badań zawartych w pracy. Krill w sposób płynny integruje się także z programem Routinator.

3.4 Routinator

Routinator jest oprogramowaniem pełniącym rolę serwera weryfikującego, który łączy się z Krill i jest pośrednikiem między ruterami a serwerem publikacji w procesie weryfikacji obiektów ROA i ASPA. Podobnie jak Krill, narzędzie to jest rozwijane przez NLnet Labs i zapewnia z nim pełną kompatybilność, co upraszcza proces uruchomienia infrastruktury RPKI w laboratorium.

Rozdział 4

Implementacja topologii

4.1 Projekt sieci

4.2 Konfiguracja

Każdy ruter użyty w topologii korzysta z przygotowanego systemu Ubuntu 22.04 LTS. Proces dodawania obrazu przeprowadzony został w konsoli głównej EVE-NG i polegał na pobraniu wymaganych narzędzi i obrazu chmurowego, włączeniu logowania do konta root oraz wstrzyknięciu hasła. Konieczna była też zmiana nazwy obrazu na wymaganą przez EVE-NG nazwę *hda.qcow2*. Ostatnim krokiem było dodanie pamięci do węzłów i naprawa uprawnień w systemie. Cały przebieg został przedstawiony na listingu 4.1.

```
apt update && apt install -y libguestfs-tools

mkdir -p /opt/unetlab/addons/qemu/linux-ubuntu-bgp
cd /opt/unetlab/addons/qemu/linux-ubuntu-bgp

wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-
cloudimg-amd64.img

virt-customize -a jammy-server-cloudimg-amd64.img --root-password
password:eve --run-command "sed -i 's/.*PermitRootLogin.*/
PermitRootLogin yes/' /etc/ssh/sshd_config"

mv jammy-server-cloudimg-amd64.img hda.qcow2

cd /opt/unetlab/addons/qemu/linux-ubuntu-bgp
qemu-img resize hda.qcow2 +5G
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

LISTING 4.1: Przygotowanie obrazu systemu

Tak przygotowany system wykorzystano następnie do utworzenia pierwszego węzła, któremu przydzielono 768MB pamięci RAM oraz 1 rdzeń procesora wirtualnego. Po jego uruchomieniu przystąpiono do instalacji oprogramowania *FRRouting*.

Ze względu na konieczność wdrożenia względnie nowego mechanizmu ASPA, zrezygnowano z instalacji prekompilowanych pakietów na rzecz samodzielnej kompilacji wersji deweloperskiej. W pierwszej kolejności pobrano i zainstalowano niezbędne narzędzia oraz skompilowano bibliotekę RTRlib. Etap ten został przedstawione na listingu 4.2.

```
curl -s https://deb.frrouting.org/frr/keys.asc | tee /usr/share/
keyrings/frrouting.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/frrouting.asc] https://deb.
frrouting.org/frr $(lsb_release -s -c) frr-stable | tee /etc/apt/sources
.list.d/frr.list

apt install -y git autoconf automake libtool make cmake pkg-config \
gcc g++ bison flex libc-ares-dev libjson-c-dev libsystemd-dev \
libpython3-dev python3-dev python3-pytest python3-sphinx \
libcap-dev libelf-dev libreadline-dev libssh-dev \
protobuf-c-compiler libprotobuf-c-dev libpcre2-dev libyang2-dev

cd /usr/src
git clone https://github.com/rtrlib/rtrlib.git
cd rtrlib
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=/usr ..
make -j$(nproc)
make install
ldconfig
```

LISTING 4.2: INSTALACJA ZALEŻNOŚCI ORAZ BIBLIOTEKI RTRLIB

Kolejnym krokiem było utworzenie użytkownika i grup systemowych i pobranie najnowszego kodu źródłowego pakietu FRRouting oraz jego kompilacja. W tym kroku konieczne było przekazanie parametru `--enable-rpki`. Wykonane polecenia znajdują się na listingu 4.3.

```
groupadd -r -g 92 frr || true
groupadd -r -g 85 frrvty || true
adduser --system --ingroup frr --home /var/run/frr/ \
--gecos "FRRouting suite" --shell /sbin/nologin frr || true
usermod -a -G frrvty frr

cd /usr/src
git clone https://github.com/FRRouting/frr.git
cd frr
./bootstrap.sh
./configure \
--prefix=/usr \
--includedir=\${prefix}/include \
```

```
--bindir=\${prefix}/bin \
--sbindir=\${prefix}/lib/frr \
--libdir=\${prefix}/lib/frr \
--libexecdir=\${prefix}/lib/frr \
--sysconfdir=/etc \
--localstatedir=/var \
--with-moduledir=\${prefix}/lib/frr/modules \
--enable-configfile-mask=0640 \
--enable-logfile-mask=0640 \
--enable-multipath=64 \
--enable-user=frr \
--enable-group=frr \
--enable-vty-group=frrvty \
--with-pkg-git-version \
--enable-rpki

apt install -y texinfo gcc-11-plugin-dev
make -j$(nproc)
make install
ldconfig
```

LISTING 4.3: TWORZENIE UŻYTKOWNIKÓW I KOMPILACJA FRROUTING

Ostatnim krokiem w tworzeniu węzła była konfiguracja systemu przedstawiona na listingu 4.4. Polegała ona na utworzeniu odpowiedniej struktury katalogów na logi oraz skopiowaniu plików konfiguracyjnych demona. W pliku `daemons` aktywowano proces `bgpd` z parametrem `-M rpki`, który jest odpowiedzialny za załadowanie modułu walidacji kryptograficznej podczas startu usługi. Odblokowano także mechanizm przekazywania pakietów IPv4 w jądrze systemu Linux, po czym zarejestrowano i uruchomiono usługę.

```
install -m 775 -o frr -g frr -d /var/log/frr
install -m 775 -o frr -g frrvty -d /etc/frr
install -m 640 -o frr -g frrvty tools/etc/frr/vtysh.conf /etc/frr/vtysh
.conf
install -m 640 -o frr -g frr tools/etc/frr/frr.conf /etc/frr/frr.conf
install -m 640 -o frr -g frr tools/etc/frr/daemons.conf /etc/frr/
daemons.conf
install -m 640 -o frr -g frr tools/etc/frr/daemons /etc/frr/daemons
install -m 644 tools/frr.service /etc/systemd/system/frr.service

sed -i 's/bgpd=no/bgpd=yes/' /etc/frr/daemons
sed -i 's/^bgpd_options=.*/bgpd_options="  -A 127.0.0.1 -M rpki"/' /etc/
frr/daemons

sed -i 's/#net.ipv4.ip_forward=1/net.ipv4.ip_forward=1/' /etc/sysctl.
conf
sysctl -p

systemctl daemon-reload
```

```
systemctl enable frr
systemctl restart frr
```

LISTING 4.4: KONFIGURACJA PLIKÓW ORAZ USŁUG SYSTEMOWYCH

Tak skonfigurowany węzeł stanowił bazę dla wszystkich ruterów w topologii. W celu uniknięcia powtarzania procesu instalacji, węzeł został przygotowany do klonowania. Konieczne do tego było wyczyszczenie zawartości pliku */etc/machine-id*. Wymusza to na systemie wygenerowanie nowego, unikalnego identyfikatora przy pierwszym uruchomieniu każdej sklonowanej maszyny. Po wyczyszczeniu pliku węzeł został wyłączony.

```
truncate -s 0 /etc/machine-id
poweroff
```

LISTING 4.5: PRZYGOTOWANIE SYSTEMU DO KLONOWANIA

Architektura EVE-NG zakłada, że zmiany na uruchomionych węzłach nie są aplikowane bezpośrednio do głównego onrazu, lecz są zapisywane w plikach tymczasowych wewnątrz katalogu powiązanego z danym laboratorium. Aby utrwalić konfigurację bazową stanowiącą wzorzec dla reszty węzłów, z poziomu konsoli serwera EVE-NG wykonano operację `comit`. Polecenie to przenosi dane z tymczasowego dysku węzła do oryginalnego pliku obrazu.

```
cd /opt/unetlab/tmp/0/<lab-id>/<node-id>
qemu-img commit hda.qcow2
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

LISTING 4.6: UTRWALENIE OBRAZU BAZOWEGO W EVE-NG

Powyższe kroki pozwoliły na powielanie węzła. Proces klonowania kolejnych ruterów został przeprowadzony bezpośrednio z poziomu interfejsu graficznego środowiska EVE-NG.

Rozdział 5

Demonstracja podatności

5.1 Route Leak

Wycieki

5.1.1 Type 1 leak

Do wytworzenia wycieku tras pierwszego typu wykorzystany został ruter R7. Węzeł ten połączony jest z dwoma operatorami w architekturze multi-homed, ale nie filtruje prefiksów rozsyłanych do ruterów R3 i R4. W efekcie ogłasza trasy otrzymane od jednego z dostawców do drugiego. Skonfigurowana na obu ruterach operatorów polityka sterowania ruchem, która przypisuje najwyższą wartość atrybutu `Local Preference` trasom pochodzącym od klientów, sprawia, że AS 65102 staje się dla nich tranzytowym systemem autonomicznym.



Rysunek 5.1: Przykładowy wpis w tablicy rutingu na ruterze R3 podczas wycieku typu 1

Wpis widoczny na rysunku 5.1 obrazuje decyzje rutingowe po wystąpieniu wycieku. Ruter R3 preferuje wysłanie ruchu kierowanego do sieci 192.168.4.0/24 przez swojego klienta R7, zamiast wykorzystać bezpośrednią ścieżkę do R4.

5.1.2 Type 2 leak

Wyciek typu drugiego opiera się na błędnej propagacji trasy pochodzącej od jednego peera do drugiego. Do wywołania tego zjawiska użyto rutera R3, którego polityka eksportowa pozwoliła na rozgłoszenie trasy 192.168.2.0/24 do R4. Konfiguracja na R4 nadaje trasie otrzymanej od R3 wyższą wartość atrybutu `Local Preference` niż trasie do tej samej sieci otrzymanej od swojego dostawcy R1.

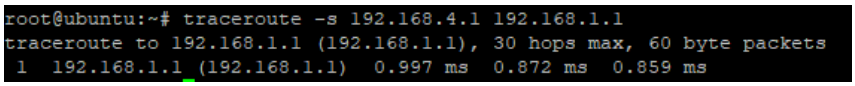


Rysunek 5.2: Przykładowy wpis w tablicy rutingu na ruterze R4 po wycieku typu 2

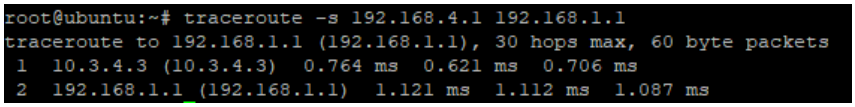
Rysunek 5.2 przedstawia decyzje rutingowe podejmowane przez ruter R4. Błędna trasa posiada atrybut `Local Preference` równy 150, co przewyższa wartość 100 przypisywaną poprawnej ścieżce. Skutkuje to preferowaniem błędnej trasy w procesie decyzyjnym BGP, czyniąc R3 nieuprawnionym punktem tranzytowym ruchu do tej sieci.

5.1.3 Type 3 leak

W scenariuszu prezentującym wyciek typu trzeciego ruter R3 rozgłasza swoim peerom trasy pochodzące od jego dostawcy nadrzędnego R1. Polityka wejściowa węzłów R2 i R4 przypisuje trasom odbieranym z sesji typu peer wyższą wartość atrybutu `Local Preference` niż trasom odbieranym od dostawców tranzytu.



Rysunek 5.3: Polecenie traceroute wydane na ruterze R4 przed wyciekiem typu 3



Rysunek 5.4: Polecenie traceroute wydane na ruterze R4 po wycieku typu 3



Rysunek 5.5: Przykładowy wpis w tablicy rutingu na ruterze R4 po wycieku typu 3

Rysunki 5.3 i 5.4 prezentują przebieg ścieżki pakietów wysyłanych do sieci 192.168.1.0/24 z sieci 192.168.4.0/24 odpowiednio przed i po wystąpieniu wycieku. Decyzje rutingowe prowadzące do tej sytuacji pokazuje rysunek 5.5, który zawiera wyciąg z tablicy rutingu BGP rutera R4. Wskazuje on, że trasa otrzymana od węzła R3 uzyskała wyższy priorytet względem trasy rozgłoszonej przez R1.

5.1.4 Type 4 leak

W tym przypadku ruter R3 wysyła trasę otrzymaną od peera R2 do swojego dostawcy R1. Skutkuje to pojawieniem się na tym ruterze drugiej trasy do sieci 192.168.2.0/24. Ponieważ obie trasy przyszły do AS64501 od jego klientów, obie posiadają taką samą wartość `Local Preference`

5.1.5 Type 5 leak

Do demonstracji wycieku typu piątego wykorzystany został ruter R7. Ponieważ w celach ograniczenia zasobów potrzebnych do uruchomienia topologii każdy AS składa się z jednego rutera, zamiast redystrybucji trasy do protokołu rutingu wewnętrznego i z powrotem do BGP utworzona została trasa statyczna do sieci 192.168.2.0/24 oraz włączono redystrybucję tras statycznych wewnątrz protokołu BGP.

```
ip route 192.168.2.0/24 10.3.7.3
router bgp 65102
    redistribute static
```

LISTING 5.1: KONFIGURACJA REDYSTRYBUCJI NA R7

```
*>  192.168.2.0/24   10.4.7.7                0          0 65102 ?
*                    10.1.4.1                           0 64501 65200 i
```

Rysunek 5.6: Przykładowy wpis w tablicy rutingu na ruterze R4 po wycieku typu 5

```
root@ubuntu:~# traceroute -s 192.168.4.1 192.168.2.1
traceroute to 192.168.2.1 (192.168.2.1), 30 hops max, 60 byte packets
 1  10.4.7.7 (10.4.7.7)  0.653 ms  0.578 ms  0.566 ms
 2  10.3.7.3 (10.3.7.3)  1.122 ms  1.113 ms  1.280 ms
 3  192.168.2.1 (192.168.2.1)  2.094 ms  2.083 ms  2.072 ms
```

Rysunek 5.7: Polecenie traceroute wydane na ruterze R4 po wycieku typu 5

Rysunek 5.6 zawiera tablicę rutingu na ruterze R4 po wycieku. Wpis z następnym przeskokiem przez R7 zawiera tylko numer AS 66102, w którym się znajduje, co potwierdza usunięcie oryginalnej ścieżki. W celu pokazania wpływu usunięcia informacji o długości ścieżki na decyzje rutingowe zrównano na R4 wartości `Local Preference` dla trasy otrzymanej od klienta R7 i operatora nadrzędnego R1. Bez tej modyfikacji ruter R4 preferowałby wysyłanie ruchu przez swojego klienta nawet, jeśli ścieżka zawarta w `AS_PATH` byłaby dłuższa od tej otrzymanej od dostawcy. Rysunek 5.7 pokazuje ścieżkę pakietów po wystąpieniu wycieku trasy.

5.1.6 Type 6 leak

Wyciek typu szóstego również został wykonany z użyciem rutera R7. Z powodów podanych w poprzednim podrozdziale dotyczącym wycieków typu 5, zamiast redystrybucji z protokołu rutingu wewnętrznego skorzystano z uproszczenia w postaci dodania sztucznych adresów na interfejsie Loopback rutera R7 i użycia redystrybucji tras bezpośrednio podłączonych. Administrator AS65102 w celu zarządzania ruchem wchodzącym zastosował technikę *path prepending*, polegającą na kilkukrotnym powtórzeniu numeru AS w atrybucie `AS_PATH`. Działanie to miało na celu wprowadzenie ruchu do swojego systemu autonomicznego przez AS65201.

```
interface lo
```

```
    ip address 192.168.7.1/28
    ip address 192.168.7.16/28
    ip address 192.168.7.32/28
    ip address 192.168.7.48/28
    ip address 192.168.7.64/27

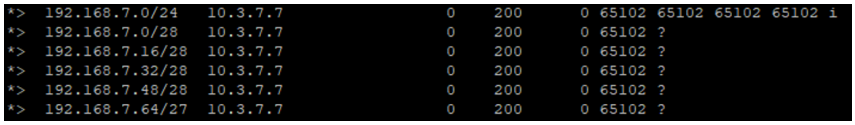
ip prefix-list PL-R7-NET seq 5 permit 192.168.7.0/24

route-map RM-TO-R3-PREPEND permit 10
    match ip address prefix-list PL-R7-NET
    set as-path prepend 65102 65102 65102

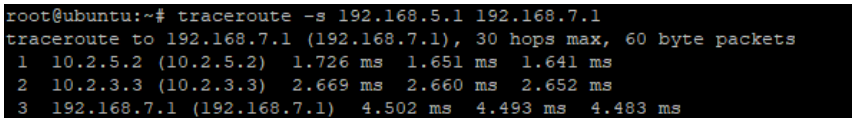
route-map RM-TO-R3-PREPEND permit 20

router bgp 65102
    bgp router-id 192.168.7.1
    network 192.168.7.0/24
    neighbor 10.3.7.3 route-map RM-TO-R3-PREPEND out
    redistribute connected
```

LISTING 5.2: Wyciek typu 6 na ruterze R7



Rysunek 5.8: Przykładowy wpis w tablicy rutingu na ruterze R3 po wycieku typu 6



Rysunek 5.9: Polecenie traceroute wydane na ruterze R5 po wycieku typu 6

Jak zauważyć można na rysunku 5.8, przy wpisie o sieci 192.168.7.0/24 preferowana jest trasa przez R4, zgodnie z zamiarem administratora AS65102. Widoczne są jednak również dokładniejsze wpisy dotyczące podsieci wewnątrz tego systemu. Kiedy otrzyma on pakiet przeznaczony do adresu należącego do jednej z widocznych podsieci, wybór trasy zależny będzie od reguły najdłuższego dopasowania (ang. *longest-prefix match*). Przykład takiej sytuacji przedstawiono na rysunku 5.9 zawierającym wynik polecenia traceroute wywołanego na ruterze R5. Ponieważ adresem docelowym był 192.168.7.1, router R3 – dysponując zaprezentowaną tablicą rutingu – skierował pakiet korzystając z wpisu 192.168.7.0/28, który został błędnie rozgłoszony przez R7. Nieuważna redystrybucja administratora spowodowała wyciek jego wewnętrznej adresacji i wpłynęła na decyzje rutingowe rutera R3.

5.2 Prefix Hijack

Celem tego podrozdziału jest przeprowadzenie i demonstracja czterech rodzajów kradzieży prefiksu

5.2.1 Exact-Prefix Hijack

Kradzież typu *Exact-Prefix* polega na rozgłoszeniu przez system autonomiczny prefiksu niebędącego jego własnością bez modyfikacji długości maski. W przypadku tego rodzaju ataku decyzje rutingowe w sieci podejmowane są na podstawie atrybutów tras propagowanych przez protokół BGP. Źródłem ataku w topologii badawczej jest ruter R6 w AS65101, rozgłaszający prefiks należący do AS 65102 w celu przejęcia ruchu skierowanego do sieci 192.168.7.0/24 i wywołania ataku *Denial of Service*. Na ruterze utworzona została trasa statyczna kierująca pakiety do wskazanej sieci na interfejs Null0 oraz rozgłoszono prefiks w procesie BGP.

```
ip route 192.168.7.0/24 Null0
router bgp 65101
    network 192.168.7.0/24
```

LISTING 5.3: Wywołanie ataku DoS dzięki kradzieży prefiksu

```
root@ubuntu:~# traceroute -s 192.168.5.1 192.168.7.1
traceroute to 192.168.7.1 (192.168.7.1), 30 hops max, 60 byte packets
 1  10.2.5.2 (10.2.5.2)  0.972 ms  0.916 ms  0.906 ms
 2  10.2.3.3 (10.2.3.3)  2.891 ms  2.882 ms  2.873 ms
 3  192.168.7.1 (192.168.7.1)  3.266 ms  3.258 ms  3.227 ms
```

Rysunek 5.10: Polecenie traceroute wydane na ruterze R5 przed kradzieżą prefiksu

```
*>  192.168.7.0/24   10.3.7.7                0    200      0 65102 i
*                    10.3.6.6                0    200      0 65101 i
```

Rysunek 5.11: Wpis w tablicy rutera R3 po przeprowadzeniu kradzieży prefiksu

```
R3# show ip bgp 192.168.7.0
BGP routing table entry for 192.168.7.0/24, version 3
Paths: (2 available, best #1, table default)
  Advertised to peers:
  10.1.3.1 10.2.3.2 10.3.4.4 10.3.6.6 10.3.7.7
  65102
    10.3.7.7 from 10.3.7.7 (192.168.7.1)
      Origin IGP, metric 0, localpref 200, valid, external, best (Older Path)
      Last update: Thu Nov  5 15:37:54 2026
  65101
    10.3.6.6 from 10.3.6.6 (192.168.6.1)
      Origin IGP, metric 0, localpref 200, valid, external
      Last update: Thu Nov  5 15:45:31 2026
```

Rysunek 5.12: Wynik procesu decyzyjnego na ruterze R3

Rysunek 5.11 pokazuje stan tablicy rutingu rutera R3 po rozgłoszeniu prefiksu przez R6. Widoczne są dwa wpisy prowadzące do atakowanej sieci: poprawny rozgłoszony

przez ruter R7 oraz podszycie się przez R6. Atrybuty dla obu tras są takie same, ale poprawna trasa została rozgłoszona wcześniej, dlatego proces decyzyjny BGP dalej ją preferuje. Decyzje rutera obrazuje rysunek 5.12, natomiast rysunek 5.10 przedstawia przebieg ścieżki pakietów przeznaczonych do sieci 192.168.7.0/24 przed udanym atakiem.

W celu demonstracji udanego ataku założono, że administrator AS 65102 używa techniki *path prepending* aby sterować ruchem wejściowym do swojego systemu autonomicznego. Zastosowana konfiguracja przedstawiona jest na listingu 5.4. Przedstawione komendy wydłużają atrybut `AS_PATH` wysyłany do sąsiada R3, aby ruch kierowany był do AS 65102 przez sąsiada R4.

```
ip prefix-list PL-R7-NET permit 192.168.7.0/24

route-map RM-TO-R3-PREPEND permit 10
    match ip address prefix-list PL-R7-NET
    set as-path prepend 65102
route-map RM-TO-R3-PREPEND permit 20

router bgp 65102
    neighbor 10.3.7.3 route-map RM-TO-R3-PREPEND out
```

LISTING 5.4: KONFIGURACJA PATH PREPENDING NA RUTERZE R7

```
*>  192.168.7.0/24   10.3.6.6                0    200      0 65101 i
*                    10.3.7.7                0    200      0 65102 65102 i
```

Rysunek 5.13: Tablica rutera R3 po udanym ataku exact-prefix hijack

```
root@ubuntu:~# traceroute -s 192.168.5.1 192.168.7.1
traceroute to 192.168.7.1 (192.168.7.1), 30 hops max, 60 byte packets
 1  10.2.5.2 (10.2.5.2)  0.749 ms  0.677 ms  0.549 ms
 2  10.2.3.3 (10.2.3.3)  1.191 ms  1.146 ms  1.091 ms
 3  * * *
 4  * * *
 5  * * *
```

Rysunek 5.14: Polecenie traceroute wydane na ruterze R5 po udanym ataku exact-prefix hijack

Zawarty na rysunku 5.13 fragment tablicy rutingu na ruterze R3 potwierdza sukces ataku. W wyniku próby inżynierii ruchu prawidłowa trasa przegrywa z trasą rozgłoszoną przez R6 w procesie decyzyjnym BGP, doprowadzając do przekierowania pakietów do atakującego i wywołując atak *Denial of Service* widoczny na rysunku 5.14.

5.2.2 Sub-prefix Hijack

W ataku typu *sub-prefix hijack* atakujący rozgłasza prefiks z dłuższą niż oryginał maską z puli należącej do innego systemu autonomicznego. Do pokazania ataku użyto

rutera R7, który rozgłosił prefiks 192.168.4.0/25 wchodzący w skład puli adresowej 192.168.4.0/24, przypisanej do AS 65201. Fragment konfiguracji odpowiedzialny za wywołanie ataku pokazany jest na listingu 5.5. Celem atakującego jest przeprowadzenie ataku MitM (ang. *Man-in-the-Middle*). Na ruterze utworzono trasę statyczną wskazującą poprawną drogę do R4 oraz rozgłoszono w protokole BGP atakowaną sieć.

```
ip route 192.168.4.0/25 10.4.7.4

router bgp 65102
    network 192.168.4.0/25
```

LISTING 5.5: Wywołanie ataku Man in The Middle dzięki kradzieży prefiksu

```
root@ubuntu:~# traceroute -s 192.168.6.1 192.168.4.1
traceroute to 192.168.4.1 (192.168.4.1), 30 hops max, 60 byte packets
 1  10.3.6.3 (10.3.6.3)  1.128 ms  0.994 ms  0.946 ms
 2  192.168.4.1 (192.168.4.1)  1.831 ms  1.792 ms  1.781 ms
```

Rysunek 5.15: Polecenie traceroute wydane na ruterze R6 przed atakiem *sub-prefix hijackk*

```
root@ubuntu:~# traceroute -s 192.168.6.1 192.168.4.1
traceroute to 192.168.4.1 (192.168.4.1), 30 hops max, 60 byte packets
 1  10.3.6.3 (10.3.6.3)  1.301 ms  1.221 ms  1.196 ms
 2  10.3.7.7 (10.3.7.7)  2.724 ms  2.711 ms  2.700 ms
 3  192.168.4.1 (192.168.4.1)  3.698 ms  3.849 ms  3.791 ms
```

Rysunek 5.16: Polecenie traceroute wydane na ruterze R6 po udanym ataku *sub-prefix hijack*

```
*>  192.168.4.0/24   10.3.4.4                0    150      0 65201 i
*                    10.1.3.1                     100      0 64501 65201 i
*>  192.168.4.0/25   10.3.7.7                0    200      0 65102 i
```

Rysunek 5.17: Fragment tablicy rutingu rutera R3 po udanym ataku *sub-prefix hijack*

Rysunek 5.17 zawiera wpisy w tablicy rutingu na ruterze R3 dotyczące sieci biorących udział w ataku. Widoczne są dwie prawidłowo rozgłoszone ścieżki od rutera R4 oraz R1 dotyczące sieci 192.168.4.0/24 oraz dodatkowy wpis otrzymany od rutera R7 obejmujący sieć 192.168.4.0/25. Zgodnie z zasadą *longest-prefix match* ruter R3 prześle ruch skierowany do pierwszej połowy adresów sieci 192.168.4.0/24 przez węzeł R7. Rysunki 5.15 oraz 5.16 ukazują ścieżki pakietów odpowiednio przed oraz po udanym ataku.

5.2.3 Forged Origin Exact-Prefix Hijack

Ataki typu *forged origin* modyfikują numer AS identyfikujący system autonomiczny, który rozgłosił prefiks. Opisywane naruszenie rutingu polega na nieautoryzowanym rozgłoszeniu cudzego prefiksu bez zmiany długości maski przy jednoczesnym podmienieniu atrybutu `AS_PATH` tak, aby podszyć się pod prawdziwego właściciela prefiksu.

W badanym scenariuszu ruter R6 rozgłasza trasę do sieci 192.168.4.0/24 oraz wykorzystuje technikę `path prepending` do doklejenia numeru AS 65201 na początek `AS_PATH`. W ten sposób `NLRI` zawarte w wiadomości *UPDATE* wygląda tak, jakby zostało stworzone przez poprawny system autonomiczny. Konfiguracja rutera R6 użyta do przeprowadzenia ataku zawarta została w listingu 5.6.

```
ip route 192.168.4.0/24 Null0

ip prefix-list PL-FORGED permit 192.168.4.0/24

route-map RM-FORGED-OUT permit 10
    match ip address prefix-list PL-FORGED
    set as-path prepend 65201
route-map RM-FORGED-OUT permit 20

router bgp 65101
    network 192.168.4.0/24
    neighbor 10.3.6.3 route-map RM-FORGED-OUT out
```

LISTING 5.6: Wywołanie ataku *forged origin exact-prefix hijack* na ruterze R6

```
*>  192.168.4.0/24   10.3.6.6              0    200      0 65101 65201 i
*                    10.3.4.4              0    150      0 65201 i
*                    10.1.3.1                   100      0 64501 65201 i
```

Rysunek 5.18: Fragment tablicy rutingu rutera R3 po ataku *forged origin exact-prefix hijack*

Rysunek 5.18 ilustruje tablicę rutingu na węźle R3. Poza dwoma poprawnymi wpisami otrzymanymi od sąsiadów R1 i R4 widnieje tam też trasa wysłana przez R6. W wyniku działania polityki rutingu na węźle R3 trasa otrzymana od klienta została oznakowana wyższą wartością `Local Preference` niż ta otrzymana od dostawcy i peera. Wpłynęło to na proces decyzyjny BGP, w wyniku którego fałszywa trasa została wybrana najlepszą, a R6 został następnym przeskokiem w drodze do sieci 192.168.4.0/24 doprowadzając do ataku *Denial of Service*.

5.2.4 Forged Origin Sub-prefix Hijack

Zasada działania ataku tego typu jest identyczna jak w poprzednim przypadku. Różnica między nimi to fakt, że w przypadku ataku *forged origin sub-prefix hijack* rozgłaszany jest prefiks o dłuższej masce niż oryginalny. Konfiguracja użyta do przeprowadzenia ataku jest tożsama z tą użytą wyżej, różni się jedynie rozgłaszaną siecią– w tym scenariuszu R6 propaguje sieć 192.168.4.0/25. Widoczna na rysunku 5.19 tablica rutingu na ruterze R3 pokazuje wpisy po przeprowadzeniu ataku. Mechanizm przechwytywania ruchu wykorzystywany w tym przypadku jest taki sam jak dla `forged origin hijack` i został opisany w podrozdziale 5.2.2.

```
*>  192.168.4.0/24   10.3.4.4                  0    150      0 65201 i
*                    10.1.3.1                       100      0 64501 65201 i
*>  192.168.4.0/25   10.3.6.6                  0    200      0 65101 65201 i
```

Rysunek 5.19: Fragment tablicy rutingu rutera R3 po ataku *forged origin sub-prefix hijack*

Rozdział 6

Implementacja i analiza zabezpieczeń

6.1 BGP Roles i atrybut Only To Customer

6.1.1 Konfiguracja i implementacja

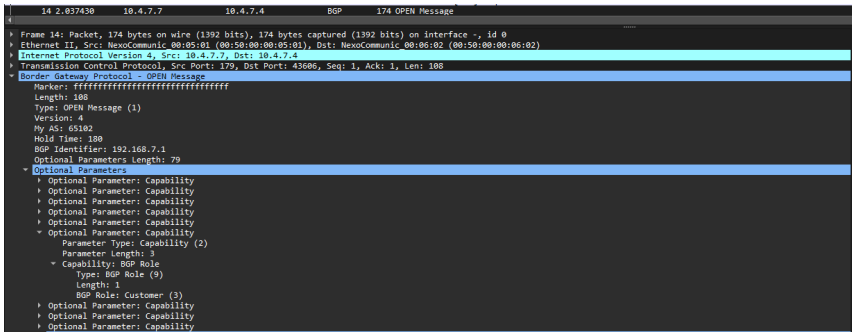
Proces konfiguracji mechanizmu *BGP Roles* opiera się na określeniu lokalnej roli rutera na sesji z sąsiadem BGP, odwzorowując istniejące relacje biznesowe między systemami autonomicznymi. Role wykorzystane w badaniu to:

- `provider` – dla sesji z klientem,
- `customer` – dla sesji z dostawcą nadrzędnym,
- `peer` – dla sesji z węzłem równorzędnym.

```
router bgp 65000
    neighbor 10.1.3.1 local-role customer
    neighbor 10.2.3.2 local-role peer
    neighbor 10.3.4.4 local-role peer
    neighbor 10.3.6.6 local-role provider
    neighbor 10.3.7.7 local-role provider
```

LISTING 6.1: Implementacja mechanizmu *BGP roles* na ruterze R3

Polecenia użyte do konfiguracji mechanizmu na ruterze R3 zostały zawarte na listingu 6.1. Pozostałe rutery w topologii zostały skonfigurowane analogicznie. Po poprawnym zaimplementowaniu *BGP Roles* rutery rozgłoszą rolę w wiadomości `OPEN` i utworzą sesję BGP z sąsiadem w przypadku zgodności ról. Proces wymiany został przedstawiony na rysunku 6.1. Poprawnie utworzona sesja zawierająca informacje o rolach widnieje na rysunku 6.2.



Rysunek 6.1: Wiadomość `OPEN` wysłana z rutera R7

```
R3# show ip bgp neighbors 10.3.7.7
BGP neighbor is 10.3.7.7, remote AS 65102, local AS 65000, external link
  Local Role: provider
  Remote Role: customer
 Description: CUSTOMER_R7_via_ens6
```

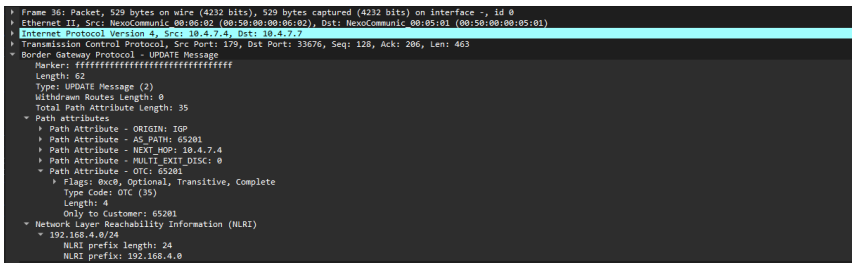
Rysunek 6.2: Sesja BGP pomiędzy ruterami R3 i R7

6.1.2 Analiza skuteczności

Mechanizm *BGP Roles* opiera się na weryfikacji poprawności rozgłoszenia trasy biorąc pod uwagę relacje biznesowe pomiędzy węzłami. Z tego powodu implementacja będzie skuteczna w przeciwdziałaniu wyciekom tras. Wpływ działania *BGP Roles* został zademonstrowany poniżej, na przykładzie wcześniej przeprowadzonych przecieków w przypadku których mechanizm ten jest skuteczny.

Przeciwdziałanie Type 1 leak

W scenariuszu wycieku typu pierwszego, ruter R7 rozgłaszał trasę otrzymaną od sąsiada R4 do swojego drugiego sąsiada R3. Po zaimplementowaniu *BGP Roles* i poprawnej konfiguracji ról lokalnych, ruter R4 oznacza trasę do sieci 192.168.4.0/24 wysyłaną do klienta R7 w wiadomości `UPDATE` atrybutem OTC (*Only To Customer*), co widać na rysunku 6.3.



Rysunek 6.3: Wiadomość `UPDATE` wysłana przez ruter R4 do R7 zawierająca atrybut OTC

```
R7# show ip bgp neighbors 10.3.7.3 advertised-routes
BGP table version is 9, local router ID is 192.168.7.1, vrf id 0
Default local pref 100, local AS 65102
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.168.7.0/24   0.0.0.0                  0             0 65102 i

Total number of prefixes 1, total number of paths 1
```

Rysunek 6.4: Trasy rozgłoszone przez ruter R7 do swojego dostawcy R3

Obecność atrybutu OTC nie pozwoli klientowi R7 na rozgłoszenie tej trasy dalej do swojego drugiego dostawcy. Trasy propagowane do rutera R3 przez R7 widoczne są na rysunku 6.4. Tablica rutingu węzła R3 zawarta na rysunku 6.5 potwierdza, że zastosowanie mechanizmu *BGP Roles* zapobiegło wystąpieniu wycieku.

```
*>  192.168.4.0/24    10.3.4.4                 0    150      0 65201 i
*                     10.1.3.1                      100      0 64501 65201 i
```

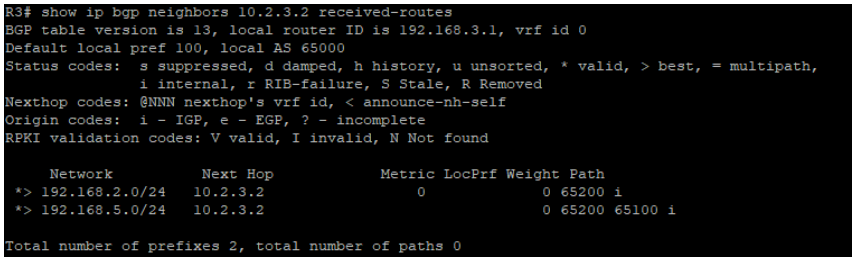
Rysunek 6.5: Trasy w tablicy rutingu R3

Przeciwdziałanie Type 2 leak

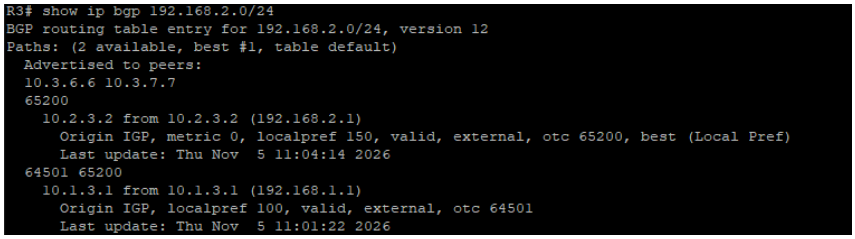
W demonstrowanym wcześniej wycieku tego typu ruter R3 błędnie rozgłosił trasę od jednego peera równorzędnego (R2) do drugiego (R4). Po konfiguracji zabezpieczenia ruter R2, wysyłając trasę przez sesję, w której role obu ruterów to *peer*, dopisuje do niej OTC. Wiadomość `UPDATE` znajduje się na rysunku 6.6.

```
▸ Frame 28: Packet, 210 bytes on wire (1680 bits), 210 bytes captured (1680 bits) on interface -,
▸ Ethernet II, Src: NexoCommunic_00:02:01 (00:50:00:00:02:01), Dst: NexoCommunic_00:01:00 (00:50:
▸ Internet Protocol Version 4, Src: 10.2.3.2, Dst: 10.2.3.3
▸ Transmission Control Protocol, Src Port: 179, Dst Port: 40726, Seq: 128, Ack: 128, Len: 144
▾ Border Gateway Protocol - UPDATE Message
    Marker: ffffffffffffffffffffffffffffffff
    Length: 62
    Type: UPDATE Message (2)
    Withdrawn Routes Length: 0
    Total Path Attribute Length: 35
  ▾ Path attributes
    ▸ Path Attribute - ORIGIN: IGP
    ▸ Path Attribute - AS_PATH: 65200
    ▸ Path Attribute - NEXT_HOP: 10.2.3.2
    ▸ Path Attribute - MULTI_EXIT_DISC: 0
    ▸ Path Attribute - OTC: 65200
  ▾ Network Layer Reachability Information (NLRI)
    ▾ 192.168.2.0/24
        NLRI prefix length: 24
        NLRI prefix: 192.168.2.0
```

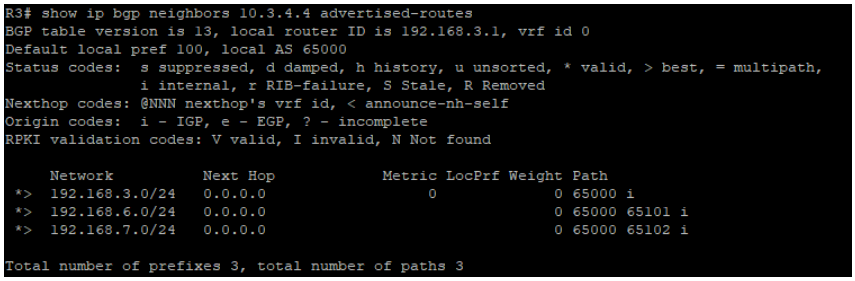
Rysunek 6.6: Wiadomość `UPDATE` wysłana przez ruter R2 do R3 zawierająca atrybut OTC



Rysunek 6.7: Trasy otrzymane przez ruter R3 od sąsiada R2



Rysunek 6.8: Podgląd tras do sieci 192.168.2.0/24 na ruterze R3

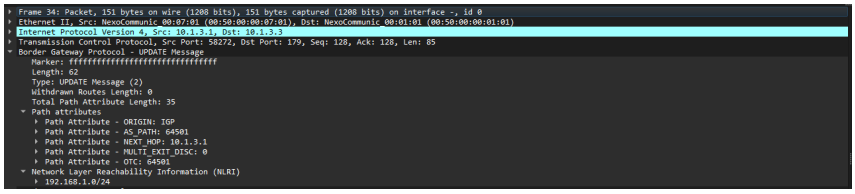


Rysunek 6.9: Trasy wysłane przez ruter R3 do sąsiada R4

Proces propagacji tras widoczny jest powyżej. Rysunek 6.7 zawiera trasy do sieci 192.168.2.0/24 i 192.168.5.0/24 otrzymane od rutera R2. Rysunek 6.8 pokazuje szczegóły dotyczące tras do sieci 192.168.2.0/24 posiadanych przez ruter R3, gdzie widoczna jest obecność atrybutu OTC otrzymanego od sąsiada R2. Rysunek 6.9 przedstawia trasy, które R3 przekazuje do R4. Brak widocznych tras do sieci 192.168.2.0/24 i 192.168.5.0/24 potwierdza działanie *BGP Roles*.

Przeciwdziałanie Type 3 leak

Przedstawiony w podrozdziale 5.1.3 wyciek polegał na rozgłoszeniu przez ruter R3 trasy otrzymanej od swojego dostawcy R1 do swoich peerów. W topologii ze skonfigurowanym mechanizmem *BGP Roles* ruter R1, wysyłając trasę do rutera R3, będącego jego klientem, oznaczy ją atrybutem OTC. Struktura wiadomości widoczna jest na rysunku 6.10. Atrybuty trasy w wiadomości `UPDATE` dotyczącej prefiksu 192.168.1.0/24 potwierdzają obecność elementu *Only To Customer*.



Rysunek 6.10: Wiadomość `UPDATE` wysłana przez ruter R1 do R3 zawierająca atrybut OTC

```
R3# show ip bgp neighbor 10.1.3.1 received-routes
BGP table version is 9, local router ID is 192.168.3.1, vrf id 0
Default local pref 100, local AS 65000
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, =
 multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *> 192.168.1.0/24   10.1.3.1                  0             0 64501 i
 *> 192.168.2.0/24   10.1.3.1                                0 64501 65200 i
 *> 192.168.3.0/24   10.1.3.1                                0 64501 65000 i
 *> 192.168.4.0/24   10.1.3.1                                0 64501 65201 i
 *> 192.168.5.0/24   10.1.3.1                                0 64501 65200 65100
i
 *> 192.168.6.0/24   10.1.3.1                                0 64501 65000 65101
i
 *> 192.168.7.0/24   10.1.3.1                                0 64501 65201 65102
i

Total number of prefixes 7, total number of paths 0
```

Rysunek 6.11: Trasy otrzymane przez ruter R3 od sąsiada R1

```
R3# show ip bgp 192.168.1.0/24
BGP routing table entry for 192.168.1.0/24, version 9
Paths: (1 available, best #1, table default)
  Advertised to peers:
  10.3.6.6 10.3.7.7
  64501
    10.1.3.1 from 10.1.3.1 (192.168.1.1)
      Origin IGP, metric 0, localpref 100, valid, external, otc 64501, best (First path received)
      Last update: Thu Nov  5 11:29:08 2026
```

Rysunek 6.12: Podgląd tras do sieci 192.168.1.0/24 na ruterze R3

```
R3# show ip bgp neighbors 10.3.4.4 advertised-routes
BGP table version is 9, local router ID is 192.168.3.1, vrf id 0
Default local pref 100, local AS 65000
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.168.3.0/24   0.0.0.0                  0             0 65000 i
 *>  192.168.6.0/24   0.0.0.0                                0 65000 65101 i
 *>  192.168.7.0/24   0.0.0.0                                0 65000 65102 i

Total number of prefixes 3, total number of paths 3
```

Rysunek 6.13: Trasy wysłane przez ruter R3 do sąsiada R4

Na rysunku 6.11 widoczne są trasy otrzymane od rutera R1. Ponieważ ruter R3 jest jego klientem, otrzymał on trasy do wszystkich sieci znanych przez węzeł R1, w tym trasę do sieci 192.168.1.0/24, która była przedmiotem wycieku w badanym scenariuszu. Rysunek 6.12 obrazuje szczegóły tras do sieci 192.168.1.0/24 na ruterze

R3. Przy wpisie otrzymanym od dostawcy R1 widnieje informacja o obecności atrybutu *Only To Customer*. Rysunek 6.13 pokazuje wynik decyzji rutera R3 odnośnie tras, które powinien rozgłosić swojemu peerowi R4. Widoczne są tam tylko trasy, które powinny być przez niego rozgłoszone. Dodany atrybut powstrzymał przekazanie trasy do sieci 192.168.1.0/24.

Przeciwdziałanie Type 4 leak

Pokazany wyciek typu czwartego powstał w wyniku rozgłoszenia przez R3 trasy otrzymanej od peera R2 do dostawcy R1

6.2 RPKI

RPKI

6.3 ASPA

ASPA

6.4 Filtracja prefiksów

Filtracja

Rozdział 7

Wnioski

Rozdział 8

Podsumowanie

Bibliografia

Dodatek A

Źródła pakietu Clicka FAMTAR